AI MEETING INTELLIGENCE TOOL

Complete Project Specification

Python | Claude API | Pydantic | FastAPI | WebSockets

Estimated build time: 3-4 days

Table of Contents

1. Project Overview

The AI Meeting Intelligence Tool is a backend AI system that accepts raw meeting transcripts (plain text, SRT subtitles, or structured JSON with speaker labels), processes them through a multi-stage extraction pipeline powered by Claude, and returns richly typed meeting artifacts: action items with assignees and deadlines, decisions with context and dissent tracking, topic segments, per-speaker sentiment analysis, and an executive summary.

Every extracted field is validated through Pydantic v2 models, ensuring type safety and dramatically reducing LLM hallucination. When the model returns malformed output, the system automatically retries with the Pydantic validation error appended to the prompt, creating a self-correction loop that pushes schema compliance above 90% on first attempt and recovers most remaining failures within 3 retries.

The system is exposed via FastAPI with both a synchronous REST endpoint and a WebSocket endpoint that streams each pipeline stage to the client in real time. A benchmark suite of 50+ hand-labeled transcripts measures precision, recall, F1, and hallucination rate to provide concrete, quantifiable results.

Key Differentiators

- **Structured output extraction** -- Pydantic models enforce typed, validated output from the LLM, eliminating free-form JSON guessing
- **Hallucination suppression** -- schema-constrained generation plus post-extraction validation reduces hallucinated fields to under 3%
- **Multi-stage pipeline** -- preprocessing, topic segmentation, action item extraction, decision extraction, sentiment analysis, and summarization run as discrete stages
- **Self-correction loop** -- validation failures trigger automatic retries with error context, so the LLM fixes its own mistakes
- **Real-time streaming** -- WebSocket endpoint streams each pipeline stage as it completes
- **Multi-format ingestion** -- accepts plain text, SRT subtitles, and structured JSON transcripts
- **Quantified benchmarking** -- tested against 50+ hand-labeled transcripts with precision, recall, F1, and hallucination rate metrics

2. Data Sources and Input Formats

The system does not record or transcribe meetings itself. It is a processing pipeline that accepts already-transcribed meeting text from external sources. Users obtain their transcripts from any of the following common sources and feed them into the tool via the API or frontend.

Where Transcripts Come From

- **Video conferencing platforms** -- Zoom, Google Meet, and Microsoft Teams all generate automatic meeting transcripts. Users export these as .txt, .srt, or .vtt files from the platform's recording or meeting history section
- **Transcription services** -- Dedicated services like Otter.ai, Fireflies.ai, Rev, and Tactiq produce speaker-labeled transcripts. These are typically exportable as plain text, SRT, or JSON
- **Manual transcription** -- For meetings without automatic transcription, a user or assistant types up the conversation. The tool accepts plain text with 'Speaker: text' line formatting
- **Speech-to-text APIs** -- If the user has raw audio, they can run it through any speech-to-text service (such as Whisper, Deepgram, or AssemblyAI) themselves, then feed the resulting text into this tool. Audio processing is outside the scope of this project
- **Calendar and productivity integrations** -- In a production context, transcripts could be pulled automatically from integrations with Zoom, Google Workspace, or Slack Huddles via their APIs

Supported Input Formats

The preprocessor auto-detects the format from the content. No manual format selection is required on the API side (though the frontend offers a dropdown hint).

Plain Text -- One utterance per line in 'Speaker: text' format. This is the simplest format and works for manually typed or basic exported transcripts.

SRT (SubRip Subtitle) -- The standard subtitle format with sequential index, timestamp range, and text content. Commonly exported from Zoom, Google Meet, and YouTube. The preprocessor strips the index numbers and timestamp lines, extracting only the spoken text. If speaker names are embedded in the text (as Zoom does), they are parsed out.

Structured JSON -- A JSON array of objects, each with 'speaker' and 'text' fields, and an optional 'timestamp' field. This is the richest format and maps directly to the internal Utterance model. Transcription services like Otter.ai and AssemblyAI can export in this format.

What the Preprocessor Does With Input

Regardless of the source format, the preprocessor normalizes all input into a clean Transcript Pydantic model (a list of Utterance objects). It resolves speaker aliases using a mapping provided in the request (for example, mapping 'JD' to 'John Doe'), strips filler markers like [inaudible] and [crosstalk], removes duplicate whitespace, and chunks long transcripts into overlapping segments if they exceed the context window limit.

3. Technology Stack

Each technology was chosen for a specific reason. The stack is minimal by design -- no unnecessary frameworks, no ORMs, no frontend build tools.

Component	Technology	Why This Choice
Language	Python 3.11+	Native type hints, async/await, union types for Pydantic
LLM	Claude API (Sonnet)	Strong structured output adherence, large context window for long transcripts
Validation	Pydantic v2	Schema enforcement, type coercion, field constraints, clear error messages for self-correction
Web Framework	FastAPI	Native async, WebSocket support, automatic OpenAPI docs, Pydantic integration
ASGI Server	Uvicorn	High-performance async server for FastAPI
Streaming	WebSockets	Bi-directional real-time communication for pipeline stage streaming
Fuzzy Matching	rapidfuzz	Fast fuzzy string matching for benchmark evaluation against ground truth
Config	python-dotenv	Environment variable management for API keys and settings
Testing	pytest + pytest-asyncio	Async-aware test framework for the full async pipeline

4. Architecture and Data Flow

The system follows a pipeline architecture where raw transcript input flows through a series of discrete processing stages, each producing validated Pydantic output, before being assembled into a final report. The architecture is intentionally linear with one parallel fan-out in the middle.

4.1 High-Level Flow

1. **Input** -- Client sends a transcript via POST /analyze (REST) or ws://host/ws/analyze (WebSocket)
2. **Preprocessing** -- The preprocessor auto-detects the format (plain text, SRT, or JSON), normalizes speaker names via an alias map, strips filler artifacts, and chunks long transcripts into overlapping segments
3. **Parallel Extraction** -- Four extractors run concurrently: action items, decisions, topics, and sentiment. Each sends a prompt with the full Pydantic schema to Claude, receives JSON, validates it, and retries on failure
4. **Summary Generation** -- The summary extractor runs after the parallel stage because it benefits from the topic and action item results as additional context
5. **Post-Extraction Validation** -- A semantic validator cross-references all extracted data against the original transcript, checking assignee names against the speaker list, verifying source quotes exist, and computing a hallucination rate
6. **Assembly** -- All stage results are assembled into a MeetingIntelligenceReport (the top-level Pydantic model) with metadata: pipeline latency, model used, and total tokens consumed
7. **Response** -- REST returns the full report as JSON. WebSocket streams each stage as it completes, then sends the final assembled report

4.2 Self-Correction Loop (Inside Each Extractor)

Every extractor follows the same internal cycle. The extractor sends a prompt containing the transcript and the JSON schema of the target Pydantic model to Claude with temperature 0. Claude returns raw text. The extractor strips any markdown fences or preamble, attempts to parse the JSON, and validates it against the Pydantic model. If validation succeeds, the result is returned. If it fails, the Pydantic error message is appended to a retry prompt and sent back to Claude. This loop runs up to 3 times. The key insight is that Pydantic errors are specific and actionable (for example, 'description must be at least 5 characters'), so the LLM can fix its output precisely.

4.3 Parallel vs Sequential Stages

Action item extraction, decision extraction, topic segmentation, and sentiment analysis are independent of each other -- they all read the same preprocessed transcript and produce separate output models. These four run concurrently using asyncio.gather to minimize total latency. The summary extractor runs sequentially after them

because it uses the extracted topics and action items as additional context to produce a higher-quality summary. Post-extraction validation runs last because it needs all extractor outputs.

5. Folder Structure

The project follows a clean modular layout. Models, extractors, and API are separated into distinct packages. The benchmark suite and tests live alongside the application code.

Path	Purpose
`meeting-intel/`	Project root
`app/`	Core application package
`main.py`	FastAPI app, REST + WebSocket endpoints
`pipeline.py`	Orchestrates the full extraction pipeline
`llm.py`	Claude API client wrapper with retry logic
`preprocessing.py`	Multi-format transcript parser and normalizer
`validators.py`	Post-extraction semantic validation
`models/`	Pydantic model definitions
`transcript.py`	Input transcript and utterance models
`action_items.py`	Action item extraction models
`decisions.py`	Decision extraction models
`topics.py`	Topic segmentation models
`sentiment.py`	Sentiment analysis models
`summary.py`	Meeting summary models
`extractors/`	LLM-powered extraction modules
`base.py`	Base extractor class with shared retry logic
`action_extractor.py`	Action item extraction
`decision_extractor.py`	Decision extraction
`topic_extractor.py`	Topic segmentation
`sentiment_extractor.py`	Sentiment analysis
`summary_extractor.py`	Summary generation
`benchmark/`	Evaluation suite
`transcripts/`	50+ test transcript JSON files
`labels/`	Hand-labeled ground truth files
`run_benchmark.py`	Runs all transcripts, computes metrics
`results.json`	Benchmark output

Path	Purpose
`frontend/`	Browser-based demo UI
`index.html`	Single-file HTML + JS frontend
`tests/`	Test suite
`test_models.py`	Pydantic model unit tests
`test_extractors.py`	Extractor unit tests
`test_pipeline.py`	Pipeline integration tests
`test_api.py`	API endpoint tests
`.env`	API keys (never committed)
`.env.example`	Template with blank values
`requirements.txt`	Python dependencies
`README.md`	Setup, architecture, benchmark results

6. Pydantic Models -- Structured Output Schema

These models are the backbone of the entire system. The LLM is instructed to return JSON matching these schemas exactly. Pydantic validates the response and rejects malformed output, which triggers the self-correction retry. Every field has explicit constraints.

7.1 Utterance and Transcript (Input Models)

An Utterance represents a single spoken segment with a required speaker name, required text content, and an optional timestamp. A Transcript is a list of Utterances plus an optional meeting date and the original raw text for validation purposes. The min_length=1 constraint on speaker and text ensures the LLM never returns empty strings.

7.2 ActionItem and ActionItemList

Each ActionItem has a description (5-500 characters, with a custom validator rejecting generic placeholders like 'TBD'), an optional assignee, an optional due_date that Pydantic coerces from string to a proper date object, a priority enum (high/medium/low), a confidence score between 0.0 and 1.0, and a source_quote that must be at least 5 characters -- this field forces the LLM to ground every extraction in the actual transcript text. ActionItemList wraps a list of these items with optional extraction_notes for the LLM to explain edge cases.

7.3 Decision and DecisionList

A Decision captures what was decided (min 5 chars), who made it, the reasoning context, whether it is tentative, whether there was dissent, and a source_quote. The is_tentative boolean distinguishes firm commitments from preliminary leanings. The dissent boolean flags decisions where disagreement was voiced, which is valuable for follow-up tracking.

7.4 Topic and TopicList

Each Topic has a title (2-100 chars), a 1-2 sentence summary, a list of speakers involved, and start/end utterance indices that map back to the Transcript. Topics must be non-overlapping and cover the full transcript. TopicList requires at least one topic -- every meeting discusses something.

7.5 SentimentReport

The SentimentReport provides an overall_tone (positive/neutral/negative/mixed enum), an energy level (high/medium/low), a conflict_detected boolean, and a per-speaker breakdown. Each SpeakerSentiment captures the speaker name, their individual sentiment, and their engagement level. This gives consumers both a quick summary and detailed per-person analysis.

7.6 MeetingSummary

An auto-generated title (3-150 chars), a concise executive summary (min 20 chars), optional duration in minutes, a participant count (min 1), and 1-5 key takeaways as a string list. This model is the first thing a consumer reads, so the constraints enforce meaningful content.

7.7 MeetingIntelligenceReport (Top-Level)

The top-level model that assembles all outputs. Contains a MeetingSummary, ActionItemList, DecisionList, TopicList, SentimentReport, a ValidationReport (from the post-extraction validator), pipeline_latency_seconds (float), model_used (string), and total_tokens_used (int). This single model is what the API returns -- one JSON object with everything.

7. Feature Breakdown

7.1 LLM Client Wrapper (llm.py)

The foundation layer that all extractors call. Wraps the Anthropic Python SDK and provides a single call_structured() method that accepts a prompt, a system prompt, and a Pydantic model class. It sends the request to Claude with temperature 0, receives raw text, strips markdown fences and preamble, parses the JSON, validates against the Pydantic model, and returns the validated instance. On validation failure, it appends the Pydantic error to a retry prompt and calls Claude again, up to 3 times with exponential backoff. Every call logs prompt tokens, completion tokens, and latency for cost monitoring.

7.2 Transcript Preprocessor (preprocessing.py)

Handles multi-format ingestion and normalization. Auto-detects the input format by checking for SRT timestamp patterns, JSON structure, or falling back to plain text line parsing. Normalizes speaker names using an alias map provided in the request (for example, mapping 'JD' to 'John Doe'). Strips filler artifacts like [inaudible] and [crosstalk] markers. For transcripts exceeding the context window limit, splits into overlapping chunks with 200-token overlap to preserve continuity. Outputs a clean Transcript Pydantic model ready for extraction.

7.3 Action Item Extractor

The most critical extractor -- this is the core resume feature. Identifies every commitment, task, and to-do from the transcript. Each extracted item includes a description, assignee, due date, priority level, confidence score, and a source quote grounded in the original text. The extractor deduplicates items mentioned multiple times, merging into a single entry with the most complete fields. Items with confidence below 0.5 are flagged as uncertain.

7.4 Decision Extractor

Identifies decisions made during the meeting. Distinguishes between firm decisions ('We are going with option A') and tentative ones ('Let us lean toward A for now') via the is_tentative flag. Tracks whether anyone disagreed via the dissent boolean. Each decision includes the reasoning context and a source quote.

7.5 Topic Segmenter

Segments the transcript into coherent topic blocks. Each topic gets a descriptive title, a 1-2 sentence summary, a list of participating speakers, and start/end utterance indices. Topics are non-overlapping and collectively cover the entire transcript -- no content falls through the cracks.

7.6 Sentiment Analyzer

Produces both a meeting-level overview (overall tone, energy, conflict detection) and a per-speaker breakdown (individual sentiment and engagement level). This gives consumers actionable insight into meeting dynamics -- was it contentious, was one person disengaged, was the energy high.

7.7 Summary Generator

Generates an executive summary designed to be read first. Runs after the parallel extraction stage so it can incorporate extracted topics and action items as context, producing a more coherent summary. Auto-generates a meeting title, writes a 3-5 sentence summary, estimates duration, counts participants, and produces up to 5 key takeaways.

7.8 Pipeline Orchestrator (pipeline.py)

The central coordinator. Accepts a raw transcript string or Transcript model. Runs preprocessing first, then fans out to four parallel extractors, waits for all to complete, runs the summary extractor, runs post-extraction validation, and assembles the MeetingIntelligenceReport. Emits stage-completion events for WebSocket streaming. Handles partial failures gracefully -- if one extractor fails after retries, the others still return results with an error marker on the failed stage. Logs total pipeline latency and per-stage latency.

7.9 Post-Extraction Validator (validators.py)

A semantic validation layer that goes beyond Pydantic's structural checks. Cross-references action item assignees against the transcript's speaker list -- an assignee not found in the speakers is flagged. Checks that due dates (if present) are not in the past relative to the meeting date. Verifies that source_quote fields are approximate substrings of the original transcript using fuzzy matching. Marks any extracted field that cannot be traced to transcript content as a hallucination. Returns a ValidationReport with a warnings list and an overall hallucination_rate percentage.

8. Pipeline Stages and Flow

This table shows exactly what happens at each stage, what it takes as input, and what it produces. This is the execution order the pipeline orchestrator follows.

Stage	Input	Output	Runs
1. Preprocessing	Raw text / SRT / JSON	Transcript model (list of Utterances)	Sequential, first
2. Action Items	Transcript model	ActionItemList (validated)	Parallel with 3-5
3. Decisions	Transcript model	DecisionList (validated)	Parallel with 2,4,5
4. Topics	Transcript model	TopicList (validated)	Parallel with 2,3,5
5. Sentiment	Transcript model	SentimentReport (validated)	Parallel with 2-4
6. Summary	Transcript + topics + action items	MeetingSummary (validated)	Sequential, after parallel
7. Validation	All extractor outputs + original transcript	ValidationReport	Sequential, last
8. Assembly	All above	MeetingIntelligenceReport	Sequential, final

9. API Endpoints and WebSocket Protocol

10.1 REST Endpoints

Method	Endpoint	Description
POST	/analyze	Accepts transcript text or JSON with optional speaker aliases and meeting date. Returns the full MeetingIntelligenceReport as JSON.
POST	/analyze/upload	Accepts a .txt, .srt, or .json file upload. Preprocesses the file content, then runs the full pipeline.
GET	/health	Returns HTTP 200 if the server is running and the Claude API is reachable.
GET	/schema	Returns the JSON schema of MeetingIntelligenceReport for API consumers to reference.

10.2 Request Model

The /analyze endpoint accepts a JSON body with a required transcript field (string, 10-50,000 characters), an optional meeting_date (ISO datetime), and an optional speaker_aliases dictionary mapping short names to full names. Input validation rejects empty transcripts, transcripts over the length limit, and malformed JSON with clear error messages.

10.3 WebSocket Protocol

The WebSocket endpoint at /ws/analyze enables real-time streaming. The client sends the transcript as the first message (same format as the REST request body). The server then streams a series of JSON events, one per pipeline stage. Each event has three fields: stage (the stage name, such as 'preprocessing', 'action_items', 'decisions'), status ('complete' or 'error'), and data (the stage's output). The final event has stage 'done' and data containing the full assembled MeetingIntelligenceReport. This allows the frontend to render results progressively as each extractor finishes, rather than waiting for the entire pipeline.

9.4 Error Handling

- **Input validation** -- rejects empty transcripts, transcripts over 50,000 characters, and malformed JSON with structured error responses
- **Rate limiting** -- max 10 requests per minute per IP using an in-memory dictionary with timestamp tracking
- **Timeout** -- 120-second maximum for the full pipeline, 60-second timeout per individual extractor LLM call
- **Partial failure** -- if one extractor fails after all retries, the pipeline still returns results from the other extractors with an error marker on the failed stage
- **Structured errors** -- all error responses return JSON with 'error' and 'detail' fields, never raw stack traces

10. Prompt Engineering Strategy

The system prompts are the most critical part of this project. Each extractor uses a carefully crafted system prompt that forces the LLM to return structured JSON. The quality of these prompts directly determines extraction accuracy.

10.1 Prompt Structure

Every extractor prompt follows the same template: a role declaration ('You are a meeting action item extractor'), followed by numbered rules (not paragraphs -- LLMs follow numbered instructions more reliably), followed by the complete JSON schema of the target Pydantic model, followed by a format enforcement line demanding only JSON output with no preamble or explanation.

10.2 Key Principles

- **Include the full JSON schema** -- the model must see every field name, type, and constraint. Do not rely on the model guessing the structure
- **Use numbered rules** -- they create a clear priority order and are easier for LLMs to follow than prose paragraphs
- **Negative constraints** -- explicitly state what the model must NOT do: 'Never invent action items not discussed', 'Do not assume assignees not named in the transcript'
- **Source quote enforcement** -- require every extracted item to include a source_quote that is a near-exact substring of the transcript. This is the single most effective anti-hallucination measure
- **Format enforcement** -- end every prompt with: 'Return ONLY valid JSON. No preamble. No markdown fences. No explanation.'
- **Long transcript reminder** -- for chunked transcripts, include: 'Process the ENTIRE transcript. Do not stop early or skip sections.'
- **Temperature 0** -- non-negotiable. Creative output destroys schema compliance

10.3 Self-Correction Retry Prompt

When Pydantic validation fails, the retry prompt appends the exact Pydantic error message and asks the model to fix just the JSON. The retry prompt says: 'Your previous response failed validation with this error: [error]. Fix the JSON to resolve this error. Return only the corrected JSON object, nothing else.' This works because Pydantic errors are specific and actionable -- the LLM gets feedback like 'description must be at least 5 characters' rather than a generic failure.

11. Hallucination Suppression System

Getting hallucination under 3% is the headline metric for this project. It is achieved through three reinforcing layers, not a single technique.

Layer 1: Prompt-Level Constraints

The system prompt explicitly tells the model to 'only extract information present in the transcript' and 'never invent data.' It requires a source_quote field on every extracted item, forcing the model to ground its output in actual transcript text. The schema constraint (temperature 0 plus explicit JSON schema) further limits the model's creative freedom.

Layer 2: Pydantic Structural Validation

Pydantic catches type mismatches, missing fields, out-of-range values, and constraint violations immediately. A description that is too short, a confidence score above 1.0, or a priority value not in the enum -- all caught before the data reaches the consumer. The self-correction loop fixes most of these automatically.

Layer 3: Post-Extraction Semantic Validation

The validator cross-references extracted data against the original transcript. It checks that every assignee name appears in the speaker list. It verifies that every source_quote is an approximate substring of the transcript using fuzzy matching. It checks that due dates are plausible. Anything that fails these checks is flagged as a hallucination and counted toward the hallucination rate. This triple-layer approach is what drives the rate below 3%.

12. Benchmarking and Evaluation

12.1 Test Transcript Dataset

The benchmark suite consists of 50+ meeting transcripts covering a range of meeting types, lengths, and complexity levels. Synthetic transcripts can be generated using an LLM, but the ground truth labels must be created by hand.

Category	Count	Characteristics
Standup meetings	10	Short, 3-5 speakers, clear action items
Sprint planning	8	Longer, task assignments, date commitments
Executive reviews	8	Decisions, strategic direction, mixed sentiment
Brainstorms	7	Many speakers, few firm decisions, high energy
One-on-ones	7	2 speakers, personal feedback, subtle action items
All-hands	5	Large group, announcements, Q&A; sections
Edge cases	5+	Empty transcripts, single speaker, foreign names, very long transcripts

12.2 Ground Truth Labeling

For each test transcript, a hand-labeled JSON file contains the correct action items, decisions, topics, and sentiment. This is tedious but essential -- without ground truth there is no way to measure accuracy. Each ground truth file follows the same Pydantic schema as the extractor output so comparison is direct.

12.3 Metrics

Metric	What It Measures	Target
Precision	Of extracted items, how many are real?	>= 85%
Recall	Of real items, how many were extracted?	>= 80%
F1 Score	Harmonic mean of precision and recall	>= 82%
Hallucination Rate	% of fields that cannot be traced to transcript	<= 3%
Schema Compliance	% of LLM responses that pass Pydantic on first try	>= 90%
Avg Latency	End-to-end pipeline time per transcript	<= 15 seconds
Self-Correction Rate	% of failed validations recovered via retry	>= 80%

12.4 Benchmark Runner

The benchmark runner loads all transcripts and their corresponding ground truth labels, runs each through the full pipeline, compares extracted items to ground truth using fuzzy string matching (80% similarity threshold via rapidfuzz), computes precision, recall, and F1 for each extractor, computes the overall hallucination rate from the validator output, and saves detailed results to a JSON file along with a printed summary table.

13. HTML Frontend

A single self-contained HTML file with embedded CSS and JavaScript. No React, no build step, no npm. Open it directly in a browser. This is the interview demo surface.

- A large text area for pasting transcripts with a format selector dropdown (Plain Text / SRT / JSON)
- A Submit button that opens a WebSocket connection and sends the transcript
- A results panel that progressively renders each pipeline stage as it completes
- Different visual styling per stage: preprocessing (gray), action items (amber), decisions (teal), topics (purple), sentiment (pink), summary (blue)
- Action items rendered as a sortable table with color-coded priority badges
- A 'Copy JSON' button that copies the full MeetingIntelligenceReport to clipboard
- Loading indicators per stage with elapsed time counters
- A hallucination rate badge in the validation section with green/yellow/red coloring based on threshold

Interview Demo Tip

Run the frontend with a simple Python HTTP server on port 3000 and have a sample transcript ready to paste. The demo starts the moment you hit Submit -- the progressive rendering of pipeline stages is visually impressive and demonstrates the WebSocket streaming.

14. Development Order

Follow this exact sequence. Each step must be working and verified before moving to the next.

Step	What to Build	How to Verify
1	Pydantic Models	Write all models. Test with sample data. Verify validation catches bad input and type coercion works.
2	LLM Client	Write call_structured(). Test with a simple prompt and a test model. Verify it returns a validated Pydantic instance.
3	Action Item Extractor	Test with 3 sample transcripts. Verify extracted items have valid source quotes grounded in the transcript.
4	Self-Correction Loop	Intentionally feed the LLM a prompt that produces invalid output. Verify retry triggers and the corrected response passes validation.
5	Remaining Extractors	Build decision, topic, sentiment, and summary extractors. Test each independently with sample transcripts.
6	Post-Extraction Validator	Run on extractor output. Verify it catches hallucinated assignees and ungrounded source quotes.
7	Pipeline Orchestrator	Wire all extractors together. Run end-to-end on one transcript. Verify the full MeetingIntelligenceReport is complete and valid.
8	FastAPI REST Endpoints	Build POST /analyze and GET /health. Test with curl or httpx. Verify the response matches the schema.
9	WebSocket Streaming	Add the WebSocket endpoint. Connect from a test client. Verify stage-by-stage events stream correctly.
10	Benchmark Suite	Create 50+ test transcripts with hand-labeled ground truth. Run the benchmark runner. Record all metrics.
11	HTML Frontend	Build the single-file frontend. Connect via WebSocket. Verify progressive rendering and correct styling per stage.
12	README	Write README with setup steps, architecture description, benchmark results table, and example usage.

15. Best Practices

Security

- Never commit the .env file -- add it to .gitignore before the first commit
- Do not expose raw LLM prompts through the API -- the /schema endpoint shows the output schema, not internal prompts
- Validate and sanitize all input -- strip HTML and script tags from transcript text
- Enforce the 50,000-character transcript length limit at the API layer

Prompt Engineering

- Always include the full JSON schema in the system prompt -- the model must see field names, types, and constraints
- Use numbered rules, not prose -- LLMs follow numbered instructions more reliably
- Temperature must be 0 for all extraction -- creative output destroys schema compliance
- Test each prompt with 10+ transcripts before integrating into the pipeline
- Include negative constraints: 'Never invent data not present in the transcript'

Error Handling

- Every extractor must return a result or a typed error -- never raise an unhandled exception that crashes the pipeline
- Wrap all LLM calls in try/except and return validation errors as structured messages
- Log every LLM call, response, token count, and validation result for debugging and cost tracking
- Set a 60-second timeout on individual LLM calls to prevent hanging

Code Quality

- Use type hints on all function signatures -- this project is fundamentally about type safety
- Write a docstring on every public function explaining what it takes, what it returns, and what can go wrong
- Keep each extractor file under 100 lines -- single responsibility
- Use environment variables for all configuration -- no hardcoded API keys or model names anywhere

Git Hygiene

- Commit after each working step in the development order
- Use clear commit messages: 'Add action item extractor with Pydantic validation and retry logic'
- Keep .gitignore updated: .env, venv/, __pycache__/, *.pyc, benchmark/results.json

16. Interview Talking Points

Prepared answers for common interviewer questions about this project:

"How does the structured output work?"

Each extractor sends a system prompt that includes the full JSON schema of the expected Pydantic model. The LLM returns raw JSON, which Pydantic validates. If validation fails -- wrong type, missing field, value out of range -- the system appends the Pydantic error message to a retry prompt and calls the LLM again. This retry loop runs up to 3 times. Over 90% of responses pass validation on the first attempt, and the self-correction mechanism recovers most of the rest.

"How did you get hallucination under 3%?"

Three reinforcing layers. First, the prompt explicitly says 'only extract information present in the transcript' and requires a source_quote field grounded in the original text. Second, Pydantic catches structural issues immediately. Third, a post-extraction validator cross-references assignee names against the speaker list and verifies source quotes are approximate substrings of the transcript. Anything ungrounded gets flagged.

"Why Pydantic instead of just parsing JSON?"

Raw JSON parsing tells you if the syntax is valid, but not if the data makes sense. Pydantic gives type coercion (string dates become date objects), field constraints (min/max length, value ranges), custom validators (rejecting generic descriptions), and clear error messages when something fails. Those error messages are what power the self-correction loop -- the LLM gets specific feedback like 'description must be at least 5 characters' instead of a generic failure.

"Why run extractors in parallel?"

Action items, decisions, topics, and sentiment are independent -- they all read the same transcript and produce separate output models. Running them concurrently with asyncio.gather cuts total pipeline latency by roughly 60% compared to running them sequentially. The summary extractor runs after the parallel stage because it uses topic and action item results as additional context.

"How would you make this production-ready?"

Add JWT authentication to the API endpoints. Replace in-memory rate limiting with Redis. Add a job queue for long-running extractions so the API responds immediately with a job ID. Cache results by transcript hash to avoid reprocessing identical inputs. Add structured logging with correlation IDs. Deploy behind a reverse proxy with TLS. Monitor LLM costs per request. Containerize with Docker.

17. Common Mistakes to Avoid

Do not build everything at once

Get the Pydantic models and a single extractor (action items) working end-to-end first. If you try to build all five extractors, the pipeline, and FastAPI simultaneously, you will waste hours debugging the wrong layer.

Do not skip the system prompt design

The quality of the system prompt determines whether the extractor works at all. If the model hallucinates action items, the fix is almost always in the prompt -- add more negative constraints, require source quotes, reduce ambiguity.

Do not use temperature above 0 for extraction

Creative LLM output is the enemy of structured extraction. Temperature 0 gives deterministic, consistent JSON. Any higher and you get field name variations, inconsistent enum values, and random formatting changes that break Pydantic validation.

Do not skip the benchmark suite

The benchmark is what makes this resume-worthy. 'I built an extractor' is generic. 'I built an extractor with 87% F1 and under 3% hallucination rate across 50 transcripts' is specific, measurable, and impressive.

Do not trust the LLM to always return valid JSON

Even with explicit instructions, LLMs sometimes add preamble text, markdown fences, or trailing commentary. The JSON parsing layer must strip these before attempting to parse. Always extract the JSON object from the response rather than parsing the raw string directly.

18. Production Readiness Roadmap

These are concrete next-step enhancements you can discuss in interviews or implement to strengthen the project further:

- **Authentication and multi-tenancy** -- Add JWT auth to the API; scope transcripts and results per user or organization
- **Async job queue** -- Replace synchronous pipeline execution with Celery or Dramatiq workers for long transcripts, returning a job ID immediately
- **Result caching** -- Cache pipeline output by transcript hash to avoid reprocessing identical inputs
- **Cost monitoring** -- Track token usage per request; set per-user cost limits; alert on budget thresholds
- **Model fallback** -- If Claude is unavailable or rate-limited, fall back to a secondary model with graceful degradation
- **Confidence thresholding** -- Allow API consumers to set a minimum confidence score, filtering out uncertain items from the response
- **Custom extraction schemas** -- Let users define additional extraction fields beyond the default schema for domain-specific needs
- **Webhook notifications** -- POST results to a callback URL when async processing completes
- **Docker and CI/CD** -- Containerize the application; add GitHub Actions for automated tests, linting, and benchmark regression detection
- **Observability** -- Add OpenTelemetry tracing; log every LLM call with latency, token count, and validation status for full pipeline visibility

---

This document is the complete specification. Use it as the starting context for every coding session. Follow the development order in Section 14 -- verify each step before moving on.

The project is scoped for 3-4 days. Day 1: models, LLM client, action item extractor. Day 2: remaining extractors, validator, pipeline. Day 3: FastAPI, WebSocket streaming, benchmark suite. Day 4: HTML frontend, README, final polish.

The most important single thing: get the action item extractor producing Pydantic-validated output with source quotes before touching anything else. Everything in this project is downstream of that extractor working correctly.